FASE 1 / 01

Eventia: definición del MVP

Diseñar hoy, escalar mañana

Universidad Rafael Landívar · Programación Web
Docente: Ing. Manuel Rojas · 5 de octubre de 2026

Problema: conectar a quienes publican actividades con personas que quieren descubrirlas e inscribirse. La empresa inicia con un equipo pequeño y necesita validar el uso del producto.

Valor del MVP: publicar un evento, encontrarlo y reservar una plaza con confirmación, manteniendo correctos los cupos.

Incluimos en el MVP

Funcionalidad	Alcance y justificación
Registro de usuarios	Registro, inicio y cierre de sesión. Identifica a organizadores y participantes y protege datos propios.
Publicación de eventos	Título, descripción, categoría, lugar, fecha y capacidad. Crea la oferta; solo organizadores.
Consulta y búsqueda	Catálogo, detalle, búsqueda y filtro por categoría. Permite descubrir actividades relevantes.
Inscripción	Una plaza por usuario y evento. Completa la acción principal del producto.
Control de cupos	Disponibilidad visible y reserva transaccional. Impide exceder la capacidad.
Consulta de inscripciones	Pantalla privada de inscripciones. Permite comprobar los planes confirmados.
Notificación	Confirmación interna persistida y lectura. Da certeza de una inscripción exitosa.

Dejamos para después

Pagos, QR y control de asistencia, cancelaciones y listas de espera, reportes avanzados y recomendaciones, correo/SMS y administración avanzada. Agregan reglas o dependencias que no son necesarias para validar el flujo inicial.

La actividad no indica el canal de notificación: se adopta el canal interno de Eventia. La fase 2 queda fuera de esta entrega.

FASE 1 / 02

Arquitectura inicial

Monolito modular, organizado por funcionalidades, con capas sencillas y una base de datos relacional única. Una aplicación sirve la interfaz y atiende la API.

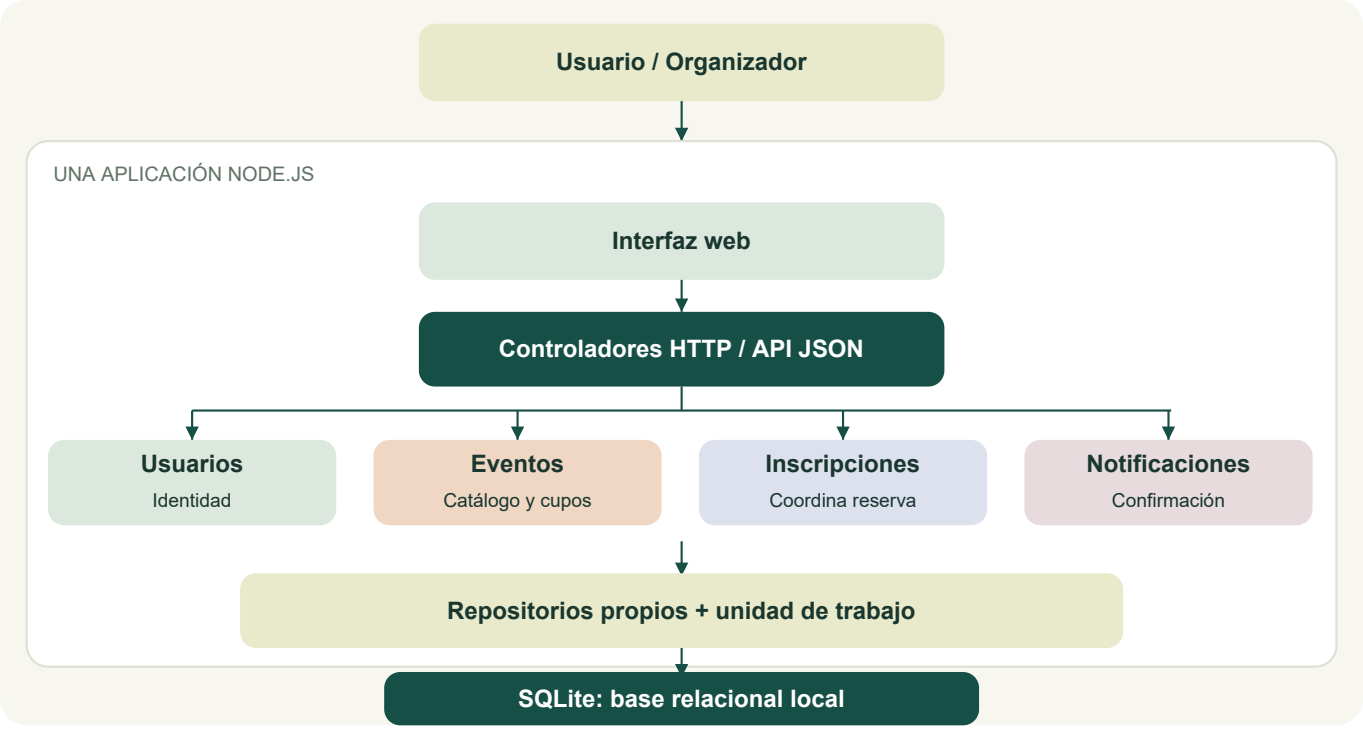


Figura 1. Arquitectura física inicial. Las relaciones entre servicios se detallan en la página siguiente. Fuente editable: docs/arquitectura.mmd y docs/arquitectura.svg.

Componente	Responsabilidad
Interfaz web	Catálogo, formularios, disponibilidad, inscripciones y notificaciones.
Controladores HTTP	Enrutar, identificar sesión, interpretar JSON y devolver errores.
Usuarios	Registro, contraseñas, identidad y sesiones.
Eventos	Publicación, búsqueda, consulta y reserva de cupos.
Inscripciones	Coordinar el flujo, impedir duplicados y consultar registros propios.
Notificaciones	Crear, consultar y marcar confirmaciones internas como leídas.
Persistencia	Repositorios por módulo y una unidad de trabajo con commit/rollback.

Justificación: responsabilidades cohesionadas, contratos internos y cambios localizados permiten evolucionar gradualmente. Un solo despliegue reduce la coordinación operativa del equipo pequeño. La clase distingue rendimiento de escalabilidad y exige evidencia antes de añadir complejidad.

FASE 1 / 03

Dependencias, datos y comunicaciones

Dependencias justificadas

Componente	Se relaciona con	Qué necesita / contrato
Inscripciones	Usuarios	Validar identidad: ensureExists(userId).
Inscripciones	Eventos	Reservar cupo y consultar estado: reservePlace y get.
Inscripciones	Notificaciones	Guardar confirmación: confirmRegistration.
Eventos	Usuarios	Identidad pública del organizador: ensureExists.
Interfaz	Controladores HTTP	Consultar y solicitar acciones mediante API JSON.

No hay dependencias circulares entre servicios. Inscripciones no modifica directamente las tablas de otros módulos. El archivo src/app.js compone e inyecta las dependencias.

Almacenamiento de datos

Tabla / propietario	Datos y reglas principales
users / Usuarios	Identidad, correo único, hash de contraseña y tipo de cuenta.
sessions / Usuarios	Hash de token, usuario y vencimiento de siete días.
events / Eventos	Información, organizador, fecha UTC, capacidad y ocupados.
registrations / Inscripciones	Usuario, evento, fecha. UNIQUE(user_id, event_id).
notifications / Notificaciones	Destinatario, inscripción única, mensaje, fecha y lectura.

Claves foráneas; capacidad entre 1 y 10,000; ocupados entre cero y capacidad. Disponibilidad = capacidad - ocupados. Contraseñas con scrypt y salt aleatorio; sesiones con cookie HttpOnly y SameSite=Strict.

Comunicaciones

Navegador/backend: HTTP local y JSON; HTTPS al desplegar. Entre módulos: llamadas síncronas dentro del proceso. Repositorios/base: SQL parametrizado sobre una conexión compartida. La notificación es interna; no hay broker ni proveedor externo.

Decisión de implementación: SQLite local sustituye a PostgreSQL para arrancar sin credenciales o instalación adicional. Se conserva el modelo relacional. No se incluye ni se afirma haber probado una migración PostgreSQL.

FASE 1 / 04

Flujo completo y control de cupos

Consultar evento → visualizar disponibilidad → inscribirse → actualizar cupo → recibir confirmación → consultar inscripción.

Paso	Comportamiento construido
1. Descubrir	El usuario consulta catálogo, busca y abre un detalle con disponibilidad.
2. Identificarse	Inicia sesión o crea una cuenta; el servidor verifica la sesión.
3. Iniciar transacción	Inscripciones usa BEGIN IMMEDIATE y comprueba duplicados.
4. Reservar	Eventos incrementa ocupados solo si hay cupo y no ha empezado.
5. Persistir	Se insertan la inscripción y la confirmación interna.
6. Confirmar	COMMIT guarda todo; cualquier fallo produce ROLLBACK completo.
7. Mostrar	Se actualiza disponibilidad y se muestran inscripción y notificación.

La disponibilidad de la pantalla es informativa. La reserva se valida nuevamente en el servidor y dentro de la transacción.

```
BEGIN IMMEDIATE;
UPDATE events SET occupied = occupied + 1
WHERE id = ? AND occupied < capacity AND starts_at > ?;
-- Debe actualizarse exactamente una fila.
INSERT INTO registrations (...) VALUES (...);
INSERT INTO notifications (...) VALUES (...);
COMMIT; -- ROLLBACK ante cualquier fallo
```

El bloque anterior es un esquema explicativo. En el código, Inscripciones coordina los contratos de Eventos y Notificaciones; los repositorios ejecutan el SQL con la misma conexión. No se espera ninguna llamada externa durante la transacción.

Criterios de aceptación

Registro y publicación funcionan; el usuario puede completar la inscripción y consultarla; recibe una confirmación persistida; duplicados y sobrecupo se rechazan; los datos son privados por usuario; un fallo revierte la operación; el estado sobrevive al reinicio.

Reflejo en el código

src/modules/registrations/service.js coordina el flujo. Eventos conserva las reglas de cupos; Notificaciones conserva la confirmación. src/infrastructure/database.js controla la transacción y database/schema.sql las restricciones.

FASE 1 / 05

Ejecución, verificación y alcance

Requisito: Node.js 24.13 o posterior dentro de la rama 24. No se necesitan paquetes npm ni credenciales externas.

```
npm.cmd run seed
npm.cmd start
# Abrir http://localhost:3000

npm.cmd run check
npm.cmd test
```

Cuenta de demostración	Correo
Organizador	organizador@eventia.demo
Participante Andrea	andrea@eventia.demo
Participante Diego	diego@eventia.demo

Contraseña de las tres cuentas: Eventia2026! Solo para demostración local. seed agrega seis eventos futuros, incluido uno de una plaza, sin borrar datos existentes.

Verificación realizada

Comprobación	Resultado
Sintaxis y tests	Sintaxis válida. 15 tests automatizados aprobados, sin fallos.
Concurrencia real	Ocho conexiones independientes para tres plazas: tres inscripciones y tres confirmaciones.
Integridad y rollback	Duplicados y sobrecupo rechazados; fallo simulado revierte cupo e inscripción.
HTTP y privacidad	Cookies, autenticación, roles, origen, límites de entrada y datos por titular verificados.
Navegador	Registro, publicación, inscripción, consulta y lectura de confirmación; persistencia tras recarga.
Diseño adaptable	Revisión de catálogo y detalle en escritorio y móvil; evidencia en docs/evidencia/.

Límites documentados

Un despliegue; SQLite con un escritor a la vez y acceso síncrono; node:sqlite experimental en Node 24.13. Confirmaciones internas, eventos gratuitos y sin cancelaciones. La entrega se verifica localmente, sin operación productiva ni fase 2.

Referencias y entregables

Manuel Rojas: clase 21 (diapositivas 10-19, 21-22) y Actividad 21 (fase 1, páginas 1-2). Documentación oficial de Node.js 24 y SQLite consultada para implementación. Martin Fowler: Monolith First. Referencias completas: docs/fase-1.md.

Propuesta editable: docs/fase-1.md. Arquitectura: docs/arquitectura.svg y .mmd. Guía: docs/demostracion.md. API: docs/api.md. Evidencia: docs/verificacion.md. Aplicación: src/ y public/. Esquema: database/schema.sql.